Lab 7 – Building a Signup Form with Validation & Good UX

1. Objective

In this lab, students will design and implement a user registration (signup) form that collects user input, validates it, and provides clear feedback to the user.

The lab reinforces the key ideas from **Module 7 – Forms & Validation**, including:

- Using Form and TextFormField for structured input
- Using FormState and GlobalKey to validate and save data
- Applying built-in and custom validation rules
- Managing focus and keyboard behaviors on mobile devices
- Improving form UX with helpful error messages and button states
- (Optional) Adding asynchronous validation before creating an account

At the end, students will have a fully functional signup form that runs correctly in:

- DartPad (Flutter)
- Android Studio
- VS Code

---

2. Requirements

This lab is structured into **four sub-tasks** inside a single project:

- **Lab 7.1 – Basic Registration Form (Form + TextFormField + Submit)**
- **Lab 7.2 – Validation Rules & Password Strength**
- **Lab 7.3 – Focus & Keyboard Management**
- **Lab 7.4 – Optional Async Validation (Email check)**

You may complete these progressively in the **same main.dart file** (or keep separate branches/versions for each step).

2.1 Functional Requirements

Your signup screen must:

1. **Collect user information**
    - Full name
    - Email
    - Password
    - Confirm password
2. **Validate user input**
    - All fields are required
    - Email must have a valid format (@ and . at minimum)
    - Password must respect strength rules:
        - Minimum 8 characters
        - At least 1 digit
    - Confirm password must match password
3. **Provide UX feedback**
    - Show inline error messages under invalid fields
    - Disable or block submission when form is invalid
    - Show a success message (e.g. SnackBar or dialog) when the form is valid
4. **(Optional – Async) Check email availability**
    - Simulate an API check (using Future.delayed)
    - If the email looks “already taken” (e.g. starts with "taken"), show an error message
    - Disable the submit button while checking

2.2 Technical Requirements

- Use **Flutter** and **Dart** only (no third-party packages).
- The entire demo must run from a single file (e.g. main.dart), so it can be pasted into **DartPad → New Flutter**.

- Use Form, TextFormField, GlobalKey<FormState>, and the validator: pattern.
- Use autovalidateMode in at least one version of the form (AutovalidateMode.onUserInteraction is recommended).
- Use FocusNode and FocusScope to move focus between fields and to dismiss the keyboard.
- Wrap the main form content in ListView or SingleChildScrollView to avoid overflow when the keyboard is open.
- Code must compile and run **without errors** in:
    - DartPad (Flutter mode)
    - Android Studio
    - VS Code

2.3 Optional Enhancements (Bonus)

You may optionally:

- Add a **Terms & Conditions** checkbox that must be accepted before submission.
- Add a **Show/Hide password** icon button.
- Add simple **password strength indicator** (e.g. Weak / Medium / Strong).
- Add a **loading indicator** in the submit button when performing async validation.
- Add extra fields such as username, phone, or role.

These are not required for passing the lab but can improve your score.

---

3. Guided Steps

Step 1 – Project Setup

1. Create a new Flutter project (e.g. flutter create lab7_forms_validation)
   or open **DartPad → New Flutter**.
2. Replace the default main.dart content with a minimal Flutter app that uses MaterialApp and a SignupScreen.
3. Verify that the app runs with a simple “Hello” text before adding any form logic.

Step 2 – Create the Basic Form Layout (Lab 7.1)

1. Inside SignupScreen, create a GlobalKey<FormState> named formKey.
2. Wrap your content with a Form widget and assign key: formKey.
3. Build the base layout:
    - Scaffold → AppBar(title: Text('Signup'))
    - body → Padding → Form → ListView (or SingleChildScrollView + Column)
4. Add the following TextFormFields:
    - Full Name
    - Email
    - Password
    - Confirm Password
5. Add a Submit button (e.g. ElevatedButton) below the fields.

Goal: At this stage, the form may not have any validation logic – it just collects input.

Step 3 – Add Basic Validation (Required Fields & Email) (Lab 7.1 → 7.2)

1. For each TextFormField, add a validator: function:
    - If the value is null or empty: return a short error message (e.g. "Name is required").
2. For the email field:
    - Check if it contains at least @ and .
    - If invalid, return "Enter a valid email".
3. In the onPressed of the Submit button:
    - Call final isValid = formKey.currentState!.validate();
    - If isValid is false, **do nothing else**.
    - If isValid is true, call formKey.currentState!.save(); and show a SnackBar confirming that the form is valid.

Goal: The form must not submit when required fields are empty or email is invalid.

Step 4 – Implement Password Rules & Confirm Password (Lab 7.2)

1. Move validation logic into separate functions (e.g. String? validateEmail(String? value)), to keep code clean.
2. For the password field:
    - Ensure at least 8 characters.
    - Ensure at least one digit (you can use RegExp(r'[0-9]')).
3. For the confirm password field:
    - Compare with the password value stored in state (e.g. _password variable).
    - If they do not match, return "Passwords do not match".
4. Enable autovalidateMode: AutovalidateMode.onUserInteraction on the Form to give quicker feedback as the user types.

Goal: The form enforces stronger password rules and ensures confirmation matches.

Step 5 – Manage Focus & Keyboard (Lab 7.3)

1. Create a FocusNode for each input field (name, email, password, confirm).
2. Assign these FocusNodes to the corresponding TextFormFields.
3. In textInputAction:
    - Set TextInputAction.next for all fields except the last one.
    - Set TextInputAction.done for the last field.
4. Use onFieldSubmitted: to move focus:
    - From Name → Email
    - From Email → Password
    - From Password → Confirm
    - From Confirm → Call _submit() / validate

5. Wrap the entire screen in a GestureDetector and in onTap, call FocusScope.of(context).unfocus() to dismiss the keyboard when tapping outside the form.
6. Ensure the form is wrapped in ListView or SingleChildScrollView to avoid overflow when the keyboard is open.

Goal: Users can move through fields smoothly with the keyboard and dismiss the keyboard easily.

Step 6 – Add Async Email Check (Optional, Lab 7.4)

1. Add a boolean bool isCheckingEmail = false;.
2. In your submit method:
    - First, run local validation:
      if (!formKey.currentState!.validate()) return;
    - If valid, set isCheckingEmail = true and call setState.
3. Simulate a server call:
    - await Future.delayed(const Duration(seconds: 2));
    - As a fake rule, treat emails starting with "taken" as already used.
4. If the email is taken:
    - Show a SnackBar or dialog: "This email is already taken".
    - Do not proceed to success message.
5. Regardless of the result, set isCheckingEmail = false and update the button:
    - While isCheckingEmail is true, disable the submit button and show a small CircularProgressIndicator instead of the text.

Goal: Demonstrate how to combine synchronous form validation with an asynchronous check before final submission.

---

4. Expected Result

When you finish the lab, your app should:

- Display a signup form with Full Name, Email, Password, and Confirm Password fields.

- Prevent submission when fields are empty or invalid.
- Show helpful error messages under each invalid field.
- Enforce password strength and ensure both password fields match.
- Allow users to navigate between fields using the Next/Done actions on the keyboard.
- Dismiss the keyboard when the user taps outside the fields.
- (Optional) Perform a fake asynchronous email check and show a message if the email is “already taken”.
- Run correctly in:
    - DartPad (Flutter mode)
    - Android Studio
    - VS Code

---

5. Submission

Deliverables:

- A zipped Flutter project **or** a DartPad Flutter share link containing the completed signup form.
- main.dart (or equivalent entry file) clearly showing:
    - Form structure
    - Validators
    - Focus/keyboard management
    - (Optional) async validation logic
- 2–3 screenshots of the app:
    - One with valid data and success message.
    - One with visible validation errors.

Evaluation Criteria:

- **Form Structure & Flow (30%)**
  Correct use of Form, TextFormField, FormState, and submit flow.

- **Validation Logic (30%)**
  Required fields, email format, password rules, confirm password, and error messages.
- **UX & Interaction (25%)**
  Focus management, keyboard behavior, inline errors, button states, and overall usability.
- **Code Quality & Clarity (15%)**
  Clean, readable code with meaningful names, separated validators, and minimal duplication.